كلية الهندسة بشبرا

FACULTY OF ENGINEERING AT SHOUBRA

Computer Organization

Single cycle processer

Group : 6

Islam Ahmed

Gehad Amr

Nada Ashraf

Nesreen Ahmed

Nour Abdel Sater

Dr. LAMIA.ALREFAAI

2025

Table of Contents

Abstract	1
Introduction	1
CPU architecture overview	1
Single cycle processer	2
Components	4
Explanation of each instruction	10
Conclusion	18

Abstract

This report presents the architecture, functionality, and operational analysis of a custom-designed 32-bit single-cycle processor. The processor supports a basic set of R-type, I-type, and J-type instructions including arithmetic, logic, branch, and jump operations. It is implemented using standard digital components and is validated through simulation.

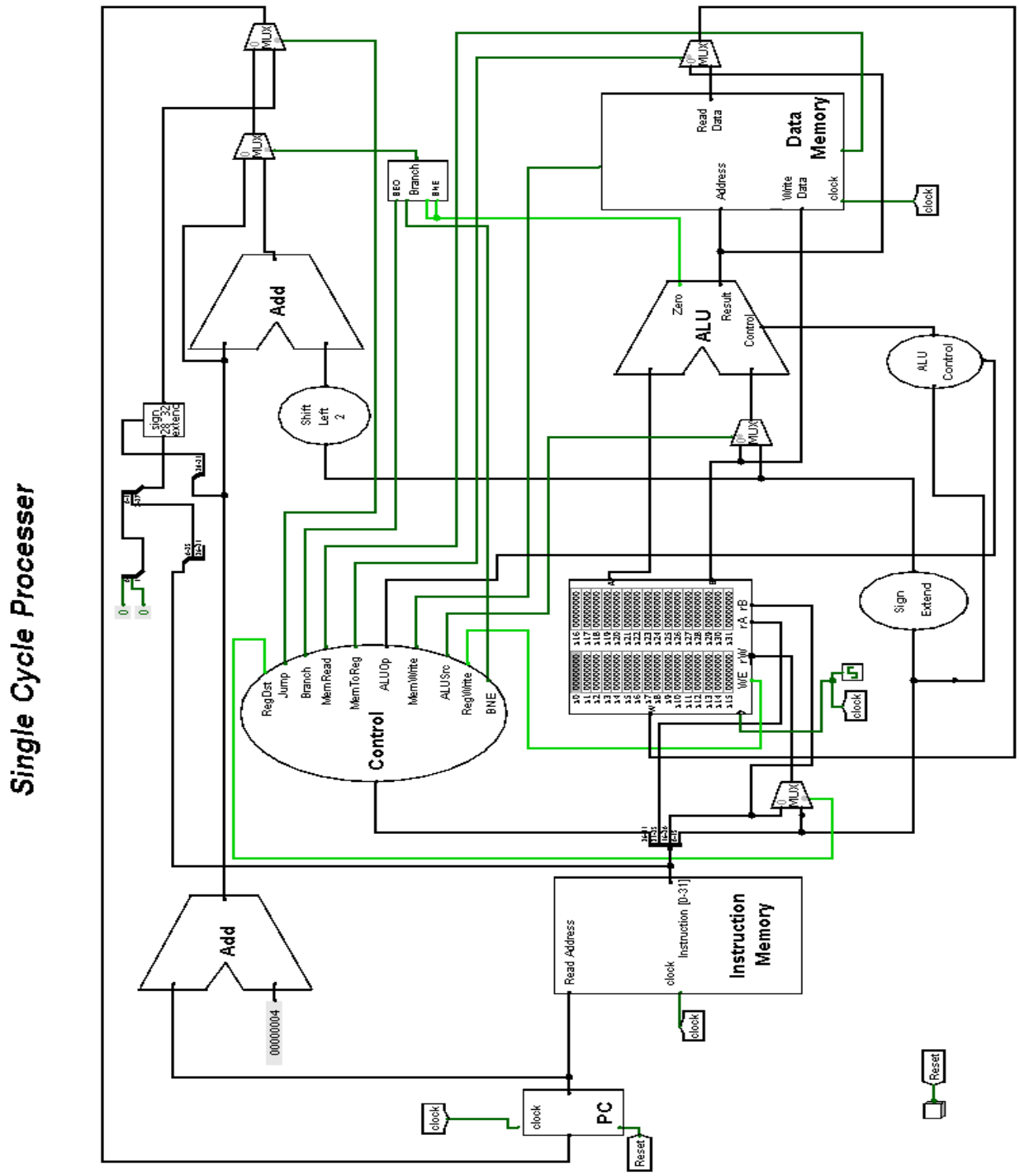
Introduction

The evolution of modern computing systems is deeply rooted in the fundamental principles of processor design. Among the core architectural models, the single-cycle processor stands out for its simplicity and direct execution model, where each instruction is completed within one clock cycle. This report explores the design and implementation of a 32-bit single-cycle processor, focusing on a minimal but effective instruction set that enables basic arithmetic, logical, comparison, branching, and control operations. The purpose of this project is to provide a practical and educational demonstration of instruction-level execution, control signal coordination, and data path design. Our processor encapsulates essential computational capabilities while serving as a foundational prototype for further enhancements such as pipelining and multi-cycle execution.

CPU architecture overview

- **Program Counter (PC)**
- **Instruction Memory**
- **Control Unit**
- **Register File (32 registers, each 32 bits)**
- **ALU (Arithmetic Logic Unit)**
- **Data Memory (if applicable)**
- **Multiplexers, Sign Extenders, etc.**

Single cycle processer



1. Definition

A **Single-Cycle Processor** is a type of CPU architecture in which **each instruction is executed entirely in a single clock cycle**. That means the processor completes all five major stages of instruction execution—**fetch, decode, execute, memory access, and write-back**—within one clock pulse. A **single-cycle processor** is a **non-pipelined** CPU design where **every instruction takes exactly one clock cycle to complete**, regardless of its complexity. This design simplifies control but may limit performance for larger, more complex instruction sets.

2. Instruction Execution

In a single cycle, the processor performs the following stages for each instruction:

Stage	Description
Instruction Fetch	The instruction is retrieved from memory using the Program Counter (PC).
Instruction Decode	The instruction is decoded, and source registers are read. Control signals are generated.
Execute	The ALU performs the necessary arithmetic or logic operation.
Memory Access	For memory instructions (like lw, sw), data is read or written to memory.
Write Back	The result is written back to the destination register.

All of these happen in **one clock pulse**, so the clock must be long enough to accommodate the **slowest instruction**.

3. Advantages

- **Simplicity**: The control unit is easier to design because there's only one cycle per instruction.
- **Predictable Timing**: Easy to estimate performance since every instruction takes one cycle.
- **Educational Clarity**: Ideal for teaching CPU design concepts.

4. Disadvantages

- **Long Clock Cycle**: The cycle must be as long as the **slowest instruction** (e.g., memory access or multiplication), leading to inefficiencies for faster instructions.
- **Poor Scalability**: As instruction sets grow more complex (e.g., floating point, multicycle memory), single-cycle designs become less practical.
- **Limited Performance**: Compared to pipelined or multi-cycle processors, it has lower throughput

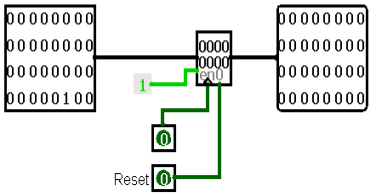
5. Use Cases

- Educational CPUs (like MIPS-based educational designs)
- Small embedded systems where simplicity is more important than speed
- Prototypes for more advanced designs

Components

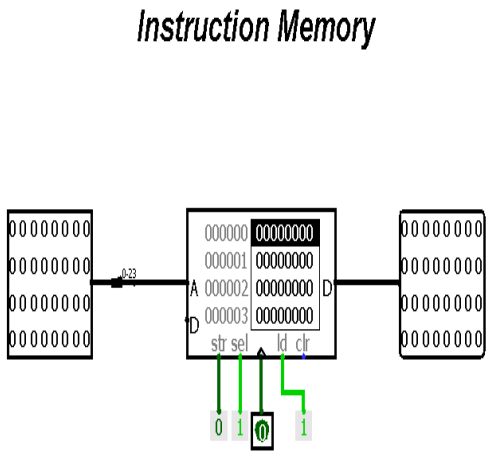
1. Program Counter (PC)

Program Counter (PC)



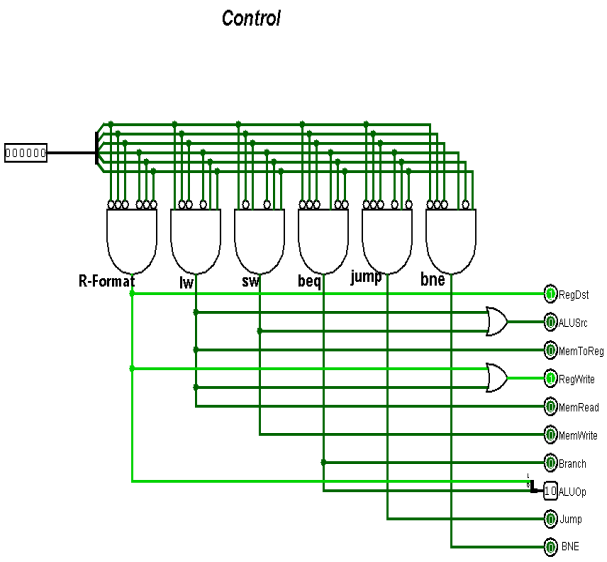
- **Function**: Stores the memory address of the next instruction to be executed.
- **Type**: 32-bit register.
- **Operation**:
  - On each clock cycle, the PC either:
    - Increments by 4 (for sequential execution), or
    - Is updated to a new address (for branches or jumps).
- **Interaction**:
  - Provides address input to the Instruction Memory.
  - Takes input from: PC + 4, Branch Adder, or Jump MUX (based on control).
- **Importance**:
  - Coordinates program flow.
  - In a single-cycle design, the PC must update correctly every cycle since there's no instruction queue or pipeline.

2. Instruction Memory



- **Function**: Holds the program code in the form of 32-bit binary machine instructions.
- **Type**: Read-only memory (ROM).
- **Operation**:
  - Accepts the address from the PC.
  - Outputs the 32-bit instruction stored at that address.
- **Interaction**:
  - Supplies the instruction to the Control Unit, Register File, and Immediate Decoder.
- **Importance**:
  - It's the primary source for fetching instructions to execute.

3. Control Unit



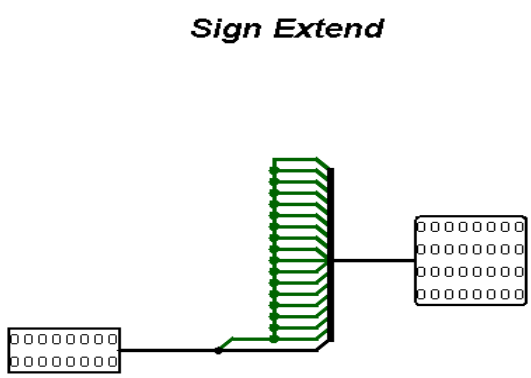
- **Function**: Interprets the opcode field of the instruction and generates control signals for the datapath.
- **Input**: Opcode (bits 31–26 of instruction).
- **Output**:
  - Signals: RegDst, ALUSrc, MemToReg, RegWrite, MemRead, MemWrite, Branch, Jump, ALUOp.
- **Interaction**:
  - Controls the MUXes, ALU, Register File, and Memory based on instruction type.
- **Importance**:
  - Acts as the brain of the processor: ensures each component reacts appropriately depending on the instruction type.

4. Register File

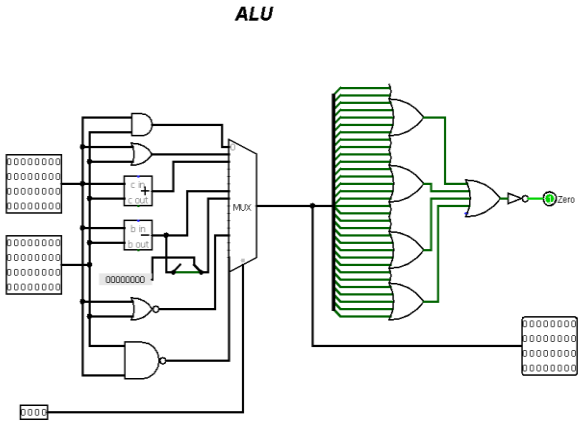
- **Function**: Stores and allows access to general-purpose registers.
- **Type**: 32 registers, each 32 bits wide.
- **Inputs**:
  - Read Register 1 (rs)
  - Read Register 2 (rt)
  - Write Register (either rd or rt, based on RegDst)
  - Write Data (from ALU or Memory, based on MemToReg)
  - RegWrite control signal
- **Outputs**:
  - Read Data 1 and Read Data 2
- **Interaction**:
  - Provides operands to the ALU and stores the result from Write Back stage.
- **Importance**:
  - Temporary storage for instruction operands and results between executions.

5. Sign-Extend Unit

- **Function**: Extends a 16-bit immediate value to a 32-bit value by replicating the sign bit.
- **Input**: Bits 15–0 of the instruction.
- **Output**: 32-bit signed value.
- **Importance**:
  - Essential for immediate-type instructions (e.g., addi, beq), allowing consistent data width during arithmetic or address calculation.

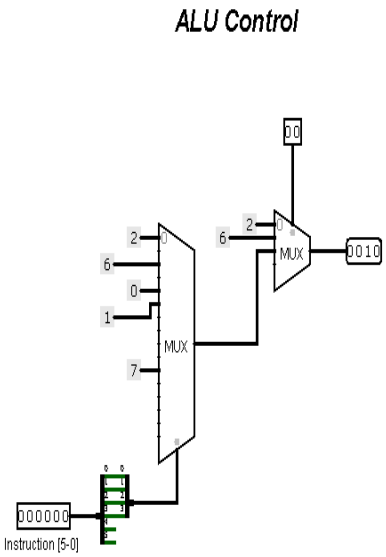


6. ALU (Arithmetic Logic Unit)



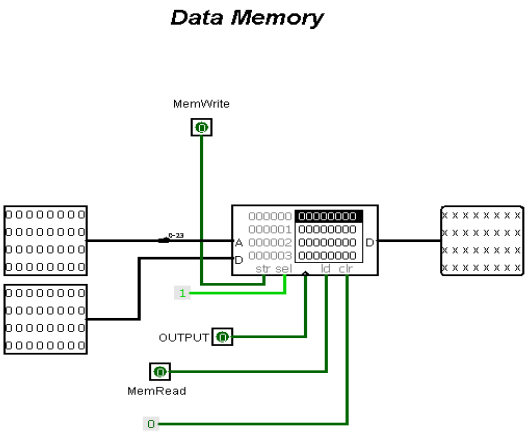
- **Function**: Performs all arithmetic and logical operations.
- **Operations Supported**: Add, Subtract, AND, OR, SLT, NOR, NAND.
- **Inputs**:
  - Operand A (always from register)
  - Operand B (from register or immediate, based on ALUSrc)
  - ALUControl signal
- **Outputs**:
  - Result
  - Zero flag (used for branch decisions)
- **Importance**:
  - Core execution unit: performs necessary calculations for all instruction types except Jump.

7. ALU Control Unit



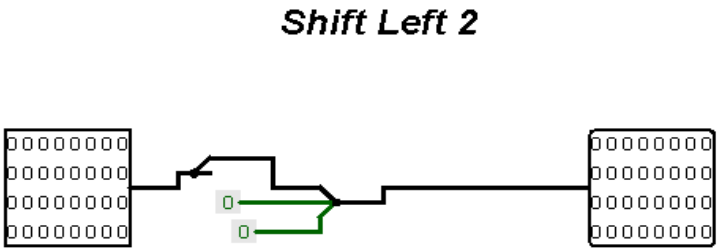
- **Function**: Converts high-level ALUOp signals from the Control Unit + Function code of R-type instructions into low-level ALUControl signals.
- **Inputs**:
  - ALUOp (2-bit signal from Control Unit)
  - Function Code (funct, bits 5–0 for R-type instructions)
- **Output**: A 4-bit ALUControl signal to drive the ALU.
- **Importance**:
  - Differentiates between operations sharing the same format (e.g., add vs. sub vs. and).

8. Data Memory



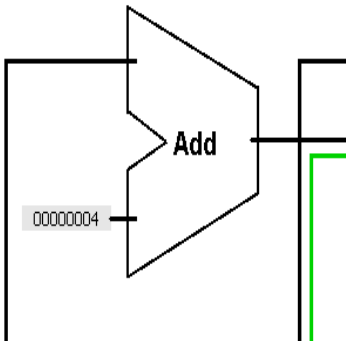
- **Function**: Used only during load (lw) and store (sw) instructions.
- **Input**:
  - Address: Calculated by the ALU.
  - Write Data: From the second register (for stores).
  - Control: MemRead, MemWrite.
- **Output**:
  - Read Data (used in load)
- **Importance**:
  - Allows the processor to interact with external data beyond registers

9. Shift Left 2 Unit



- **Function**: Shifts a 32-bit immediate value left by 2 bits.
- **Used For**:
  - Creating the effective branch offset from sign-extended immediate values.
- **Importance**:
  - Multiplies offset by 4 (since instructions are word-aligned, each word = 4 bytes).
  - Ensures proper calculation of branch targets.

10. Adders



- **Adder 1 (PC + 4)**:
  - Adds 4 to PC for sequential instruction access.
- **Adder 2 (Branch Target)**:
  - Adds PC + 4 and Shifted Immediate to compute branch target.
- **Importance**:
  - Enable both standard flow and branch flow decisions.

11. Multiplexers (MUX)

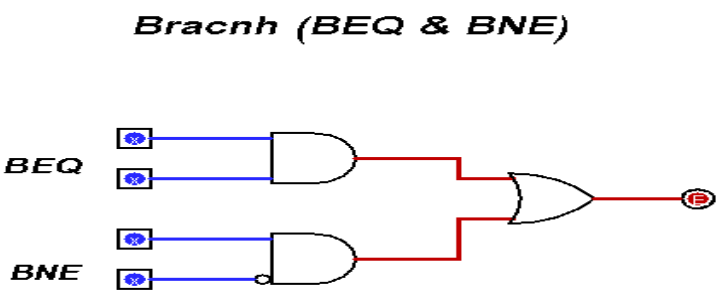
Multiplexers are used to select between multiple input sources:

MUX	Inputs	Controlled By	Function
RegDst	rt, rd	RegDst	Selects destination register
ALUSrc	Register data, Immediate	ALUSrc	Selects 2nd ALU operand
MemToReg	ALU result, Memory data	MemToReg	Selects data for register write-back
PCSrc	PC + 4, Branch Target	Branch & Zero	Determines PC update
Jump MUX	Output of PCSrc, Jump Target	Jump	Final PC value

Importance: Enable dynamic data routing based on control signals. Without them, the datapath would be fixed and inflexible.

12. Branch Control Logic

- **Function**:
  - Uses Zero flag from ALU to evaluate conditions like:
    - BEQ: Branch if ALU result is Zero.
    - BNE: Branch if ALU result is not Zero.
- **Output**: Control signal to update PC or not.
- **Importance**:
  - Enables conditional execution paths in programs.



13. Jump Logic

- **Function**:
  - Extracts bits 25–0 from instruction and shifts left 2 bits.
  - Concatenates this with top 4 bits of PC + 4 to form a full 32-bit jump address.
- **Importance**:
  - Allows for long-range jumps within the address space—critical for subroutines and flow control.

14. Clock Signal

- **Function**: Triggers simultaneous operation across all components.
- **Importance in Single-Cycle**:
  - All operations (fetch, decode, execute, memory, write-back) must complete within one clock cycle.
  - Hence, the **longest component delay** dictates the **minimum cycle time**.

Explanation of each instruction

1. ADD — Add Registers

- **Syntax:** **ADD rd, rs, rt**
- **Format**: R-type (Register type)

Field	Size	Description
opcode	6	Operation code (e.g., 000000 for R-type)
rs	5	First source register
rt	5	Second source register
rd	5	Destination register
shamt	5	Shift amount (0 for ADD)
funct	6	Function code (e.g., 100000 for ADD)

- **Datapath Steps:**

1. **Instruction Fetch**: PC fetches instruction from memory.
2. **Instruction Decode**: Control unit identifies ADD.
3. **Register Read**: Read R2 and R3 from register file.
4. **ALU Operation**: ALU adds R2 and R3.
5. **Write Back**: Result is written to R1.

- **Control Signals:**

Signal	Value	Purpose
RegDst	1	Use rd for destination
ALUSrc	0	Use register, not immediate
MemtoReg	0	Result comes from ALU
RegWrite	1	Enable writing to rd
ALUOp	10	ALU control interprets funct
MemRead	0	No memory read
MemWrite	0	No memory write
Branch	0	Not a branch instruction
Jump	0	Not a jump

2. SUB — Subtract Registers

- **Syntax:** **SUB R5, R1, R4**
- **Format**:

Same layout as ADD.

| Funct for SUB = 100010 |

- **Datapath Steps:**

Same as ADD, but ALU performs subtraction.

- **Control Signals:**

Same as ADD, except:

- ALUOp still = 10, but funct = 100010 maps to subtraction in ALU control.

3. BEQ — Branch if Equal

- **Syntax:** **BEQ rs, rt, offset**
- **Format**: I-type (Immediate)

Field	Size	Description
opcode	6	e.g., 000100 for BEQ
rs	5	First source register
rt	5	Second source register
offset	16	Signed offset for PC update

- **Datapath:**

6. **Fetch & Decode** instruction.
7. **Read Registers** R4 and R5.
8. **ALU Comparison:** R4 - R5
9. **Check Zero output:**
   - If result is zero → branch is taken.
10. **Update PC** with offset (shifted left 2 bits for word addressing).

- **Control Signals:**

Signal	Value	Purpose
ALUSrc	0	Use second register
ALUOp	01	Tells ALU to compare
Branch	1	Enable branch logic
PCSrc	Depends on ALU Zero output	

4. BNE — Branch if Not Equal

- **Syntax:** **BNE rs, rt, offset**
- **Format**: I-type

  Same as BEQ, but opcode is different (000101 for BNE).
- **Datapath Steps:**

  Same as BEQ, except: Branch taken if ALU output is **not** zero.
- **Control Signals:**

  Same as BEQ, but logic checks for Zero == 0 to take the branch.

5. JUMP — Unconditional Jump

- **Syntax:** **JUMP address**
- **Format**: J-type (Jump)

Field	Size	Description
opcode	6	e.g., 000010 for JUMP
address	26	Jump target address (word-aligned)

- **Datapath:**

1. **Instruction fetch and decode.**
2. **Control unit identifies jump.**
3. **New PC = (PC[31:28] << 28) | (address << 2)**

- **Control Signals:**

Signal	Value	Purpose
Jump	1	Jump to target
PCSrc	1	Use jump address
ALUOp	x	Don't care
RegWrite	0	No register update

6. SLT — SET ON LESS THAN

- **Syntax:** **slt rd, rs, rt**
- **Format** : Instruction Format (R-Type)

Field	Value
opcode	**000000**
rs	**source register 1**
rt	**source register 2**
rd	**destination register**
shamt	**00000**
funct	**101010 (0x2A)**

- **Datapath:**

1. **Instruction Memory:** Reads the slt instruction.
2. **Register File:** Reads values from rs and rt.
3. **ALU Control:**
   - ALUOp = 10 (for R-type).
   - funct = 101010 → ALU Control outputs the SLT operation.
4. **ALU:** Compares rs and rt. If rs < rt, result = 1, else result = 0.
5. **Write-back:** Result is written to rd register via the Register File.

- **Control Signals:**

Signal	Value
RegDst	1
RegWrite	1
ALUSrc	0
MemtoReg	0
MemRead	0
MemWrite	0
Branch	0
Jump	0
ALUOp	10

7. SW – Store Word

- **Syntax:** **SW rt, offset(rs)**

  i. `rs`: base register (holds memory address)
  ii. `offset`: 16-bit immediate value (signed)
  iii. `rt`: target/source register

- **Format**: (I-type)

Field	Bits	Description
opcode	6	100011 for LW, 101011 for SW
rs	5	Base register
rt	5	Target/source register
imm	16	Offset (signed)

- **Datapath Steps:**
  1. **Fetch instruction**
  2. **Decode and read rs and rt**
  3. **ALU calculates address**: `Address = R[rs] + offset`
  4. **Memory write**: store data from `R[rt]` to memory
- **Control Signals:**

Signal	Value	Comment
RegDst	X	Don't care (no register write)
ALUSrc	1	ALU gets second input from immediate
MemtoReg	X	Don't care
RegWrite	0	No write to register
MemRead	0	Not reading from memory
MemWrite	1	Write to memory
Branch	0	Not a branch
ALUOp	00	ALU performs addition

8. LW – Load Word

- **Syntax:** **LW rt, offset(rs)**
  i. `rs`: base register (holds memory address)
  ii. `offset`: 16-bit immediate value (signed)
  iii. `rt`: target/source register
- **Format**: (I-type)

Field	Bits	Description
opcode	6	100011 for LW, 101011 for SW
rs	5	Base register
rt	5	Target/source register
imm	16	Offset (signed)

- **Datapath Steps:**

  1. **Fetch instruction**
  2. **Decode and read rs**
  3. **ALU calculates address:** `Address = R[rs] + offset`
  4. **Memory read** at `Address`
  5. **Write** result to register `rt`

- **Control Signals:**

Signal	Value	Comment
RegDst	0	Write to rt
ALUSrc	1	ALU gets second input from immediate
MemtoReg	1	Write data from memory
RegWrite	1	Write to register
MemRead	1	Read from memory
MemWrite	0	Not writing to memory
Branch	0	Not a branch
ALUOp	00	Just add offset

9. AND — Bitwise AND

- **Syntax:** **AND rd, rs, rt**
- **Format**: R-type

Field	Bits	Description
opcode	6	000000 (R-type)
rs	5	First source register
rt	5	Second source register
rd	5	Destination register
shamt	5	Shift amount (0 here)
funct	6	100100 for AND

- **Control Signals:**

Same as ADD (R-type), but funct = 100100 → ALU performs AND.

10. NAND — Bitwise NAND

- **Syntax:** **NAND rd, rs, rt**
- **Format**: R-type
  - You need a unique `funct` code, e.g., `101100`
  - 📌 This is not in standard MIPS, but many custom CPUs support NAND logic

- **Control Signals:**
  - ALUOp = `10` → ALU uses `funct` field
  - ALU Control decodes `funct = 101100` to NAND

11. OR — Bitwise OR

- **Syntax:** **OR rd, rs, rt**
- **Format**: R-type

  | Funct = 100101 |
- **Control Signals:**

  Same as AND, but funct = 100101.

12. NOR — Bitwise NOR

- **Syntax:** **NOR rd, rs, rt**
- **Format**: R-type

  | Funct = 100111 |

Datapath Notes for All Logic Instructions:

1. **Instruction Fetch** → PC gets instruction.
2. **Decode** → Control unit reads opcode 000000.
3. **Register Read** → Get rs, rt values.
4. **ALU Logic** → ALU performs logic operation using funct.
5. **Write Back** → Result saved to rd.

Conclusion

In this report, we examined the architecture and internal workings of a 32-bit single-cycle processor in depth. A single-cycle processor executes every instruction in exactly one clock cycle, making it conceptually simple and suitable for educational purposes. We analyzed each core component — from the Program Counter and Control Unit to the ALU, memory modules, and supporting units like multiplexers and adders — detailing their roles and interactions within the processor's data path.

While the single-cycle design allows for straightforward implementation, it comes with performance limitations due to its fixed clock cycle duration, which must accommodate the slowest instruction. Despite these limitations, understanding this architecture builds a strong foundation for grasping more complex multi-cycle and pipelined processors. Through this study, we gain valuable insight into instruction execution, control signal generation, and the synchronous flow of data — all of which are essential concepts in computer organization and processor design.

Thank You ....